

Deber 2

Cambiar la tecnología sin cambiar la especificación

César Martínez · Programación Asistida de Aplicaciones · USFQ

6 de octubre de 2026

React: 36 pruebas. Flutter SDD: 18 pruebas. Spec: 100% intacta.

Cronómetro hasta primer build: 9 min 50 s. Hasta seis casos: 10 min 9 s.

Este documento reúne las seis respuestas, las mediciones, la explicación de la práctica y los inventarios de enunciados. Las salidas completas de pruebas, diffs y hashes están en respuestas.md y evidencias/ del proyecto React. La publicación se registra en publicacion.md.

Los apartados que la guía pide realizar a mano fueron generados por el agente según la solicitud y requieren revisión personal; no se atribuye autoría manual al estudiante.

Deber 2 - SDD: migración de Flutter a React

Programación Asistida de Aplicaciones - USFQ. Fecha: 6 de octubre de 2026. Trabajo preparado para César Martínez. Agente: Codex.

Resultado comprobado

React/Vite con JavaScript, una pantalla y useState, capas presentation -> domain <- data. **6/6 casos de aceptación, LSP y tres pruebas obligatorias de pantalla**; suite ampliada **36/36**. Build correcto. Flutter SDD: **18/18**, All tests passed en Chrome, con ajuste de runner para Windows. Cronómetro React: **9.83 min hasta primer build; 10.15 min hasta seis casos**. Sin tiempos históricos Flutter para comparar.

La especificación viajó intacta: **64/64 enunciados, 100%**, bajo el método publicado en analisis_spec.md (analisis_spec.md). La constitución original tiene 14 reglas idénticas, 12 adaptadas y 1 reemplazada. El plan modifica 40/54 decisiones inventariadas.

Ubicación y alcance

- Origen inmutable: rama `sdd`, commit `0e14e00ef58f9146f9c0f51ff5fadd30f599412f`; feature `07_Participacion/divisor_cuenta/specs/001-dividir-cuenta`.
- Copia Flutter de entrega: `../divisor_cuenta`, creada desde ese commit, más el ajuste de pruebas. No sobrescribe la carpeta anterior de `main`.
- React local: `divisor_cuenta_web`, repositorio propio en `main`. Constitución: `..specify/memory/constitution.md`; feature: `specs/001-dividir-cuenta`.
- Publicación conjunta: `main` del repositorio existente; la fuente React se exporta también a `../divisor_cuenta_web_entrega`, conservando el proyecto local independiente. El bundle Git acompaña su historial. No hay necesidad de crear una rama `feature`.
- `SPECIFY_FEATURE_DIRECTORY`: `specs/001-dividir-cuenta`; se configura en `../ejecutar.ps1` y queda persistido por los scripts de Spec Kit. La spec aún dice `sdd` porque identifica su origen.
- No se ejecutó `speckit-specify`. Las skills se ejecutan por el agente, no son ejecutables de shell que generen automáticamente la app. Scripts y reportes prueban las fases.

1. ¿Qué porcentaje viajó intacto, adaptado o no reusable?

Intacto **100% (64/64)**; adaptado **0%**; no reusable **0%**. Todos los enunciados inventariados describen entradas, resultados, restricciones de producto, validación o criterios ejecutables. Ninguno exige Flutter, Dart, Riverpod ni widgets. La fecha y Feature Branch=`sdd` son metadatos de origen, no decisiones de implementación; se excluyen del denominador y se conservan en el archivo.

El método cuenta enunciados atómicos por aparición, divide obligaciones independientes y no deduplica reglas repetidas entre FR, Edge Cases y SC. Otra deduplicación cambiaría el denominador, pero aquí seguiría siendo 100% reusable porque ninguna obligación depende de Flutter. $\%CÓMO = 0/64 \times 100 = 0\%$. No supera el 30%, **umbral pedagógico del deber**, no una regla universal.

Evidencia: diff inicial y final sin salida, código 0, archivos de diff de 0 bytes; comparación adicional del objeto Git inicial `6cce762` contra `sdd` idéntica; SHA256 de ambas copias igual. El tiempo 10.15 min es contexto descriptivo: no mide calidad ni demuestra que SDD sea más rápido que otro enfoque. No existe cronómetro Flutter histórico comparable.

2. Constitution: regla por regla

La tabla completa C01-C27 de analisis_spec.md (analisis_spec.md) cita cada regla original y la redacción React. Resultado: **14/27 idénticas (51.85%)**, **12/27 adaptadas (44.44%)** y **1/27 reemplazada (3.70%)**. Modificadas: 13/27. Las cifras corresponden a la Constitution original 1.0.0 de sdd, preservada en evidencias/constitution-flutter-original.md, no a la aclaración posterior del runner web en la copia Flutter.

Ejemplos: «presentation depende de domain, nunca de data» y «No guardar secretos ni API keys» permanecen idénticas. «domain no importa package:flutter ni data» se adapta a «src/domain no importa react, DOM ni data; es JavaScript puro». «main.dart es el único punto...» pasa a src/main.jsx. «redondear(double importe)» pasa al contrato aplicar(valor), con una sola operación. No desaparecen SRP, OCP, LSP, ISP ni DIP.

La regla «El SDK y sus dependencias generadas son la base; no agregar paquetes externos» se reemplaza por el stack React/Vite/Vitest/Testing Library/jsdom exigido y la prohibición de librerías externas de estado. Mantener literalmente esa prohibición impediría cumplir el deber. La constitución React es 2.0.0 por esta sustitución incompatible. La copia Flutter añade una excepción de desarrollo en 1.0.1 para package:test; no cambia producto ni reglas de negocio. No se introducen principios nuevos de producto sin equivalente; el catálogo explicativo responde a la regla de la materia.

3. ¿Se modificó algún enunciado de la spec?

No. Se conservaron los bytes de la spec y su significado. El cálculo, los mensajes, la pantalla única y los modos de redondeo no dependen de tecnología. React implementa las decisiones en plan y código. No hay commit de adaptación de spec porque no hubo adaptación. La presencia de sdd como metadato no es contradicción con trabajar en main, y el análisis lo documenta.

Los inconvenientes encontrados pertenecen al CÓMO: runner Flutter bloqueado, assets CanvasKit y herramientas Node/npm. Se resolvieron en configuración y scripts; no se alteraron los seis escenarios para conseguir pruebas verdes. El historial conserva el commit de copia 6cce762 y el plan 8731e80. Evidencia de identidad en evidencias/spec-inicial-historial.txt y evidencias/spec-identidad.txt.

4. Los seis casos en Flutter y React

Caso	Monto / personas / propina / modo	Esperado Flutter	Esperado React	Cambió
1. reparto normal	100 / 4 / 10 / exacto	27.50	27.50	No
2. sin propina	90 / 3 / 0 / exacto	30.00	30.00	No
3. cero personas	50 / 0 / 0 / exacto	Debe haber al menos una persona	Debe haber al menos una persona	No
4. monto no numerico	NaN / 4 / 0 / exacto	Monto inválido	Monto inválido	No
5. redondeo exacto	10 / 3 / 0 / exacto	3.33	3.33	No
6. redondeo hacia arriba	10 / 3 / 0 / arriba	4.00	4.00	No

No cambió entrada, valor esperado, mensaje ni escenario. double.nan de Dart se representa como NaN en JavaScript; es una traducción sintáctica, no otra regla. Una comprobación automatizada parseó casos-flutter.dart, importó casosDePrueba.js y comparó los seis objetos; evidencia comparacion-casos.json con iguales=true. Las suites verifican 27.50, 30.00, 3.33, 4.00 y los dos mensajes exactos. Las pruebas de errores comprueban también que no se invoca calcular.

La guía pide que el estudiante escriba a mano ese archivo y el análisis. En esta entrega los produjo el agente según la solicitud del usuario: **0 líneas manuales del estudiante**. Deben revisarse y explicarse personalmente; no se presenta la autoría manual como un dato observado.

5. ¿Qué partes del plan Flutter ya no sirven?

1. **Dart/Flutter y widgets**: .dart, MaterialApp y widgets se sustituyen por módulos .js, JSX y el DOM de React. El plan apunta al navegador, no a Android.
2. **setState y DivisorController por constructor**: useDivisor usa useState y recibe servicios ordinarios. PantallaDivisor importa el hook estáticamente; main.jsx compone estrategias y servicios. No se inyecta el hook como prop.
3. **flutter_test/test_api**: Vitest ejecuta dominio; Testing Library interactúa con controles accesibles en jsdom. Se añaden setupFiles y jest-dom, conservando los datos de los seis casos.
4. **Interfaz Dart redondear(double)**: JavaScript usa el contrato estructural aplicar(valor). La calculadora depende de esa operación y no reconoce clases concretas.
5. **lib/ y main.dart**: src/domain, src/data, src/presentation y main.jsx conservan la dirección de dependencias pero cambian rutas y composición.
6. **Herramientas y empaquetado**: Flutter produce APK; npm run build produce dist/ para web. Node ejecuta herramientas; no se añadió backend.

El inventario P01-P54 está en analisis_plan.md (analisis_plan.md): **40/54 decisiones modificadas**, 14 intactas. El diff físico tiene **50 líneas añadidas y 44 eliminadas**. No se equiparan estas operaciones a decisiones atómicas: la guía usa ambos términos, por eso se informan separadamente. Sobreviven cálculo puro, almacenamiento ausente, rendimiento constante, alcance y varias comprobaciones SOLID; cambia sobre todo cómo se materializan.

6. Artefacto más y menos reusable

La **spec** fue el artefacto físico más reusable: copia idéntica, diff vacío y 64/64 enunciados intactos. Los **escenarios de prueba** empatan en significado: mismos seis objetos comparados y resultados verdes. El archivo runner no viaja byte por byte porque la sintaxis cambia.

El **código de implementación** fue el menos reusable físicamente: se reescribieron módulos Dart como JavaScript/JSX y controles Flutter como DOM. Se reutilizan las fórmulas y responsabilidades, pero no los archivos ejecutables. Entre los documentos de diseño, plan y tasks son los menos portables: el inventario del plan modifica 40/54 decisiones y las tareas nombran archivos/herramientas distintos. La constitución queda en medio: preserva principios pero adapta lenguaje y una restricción de dependencias.

Esta evidencia apoya la hipótesis de separación **QUÉ/CÓMO en este repositorio**, no demuestra una superioridad universal ni una diferencia temporal contra vibe coding. Haría falta otra ejecución comparable con criterios y registro iguales para esa conclusión.

Bitácora y límites de medición

Métrica	Flutter (laboratorio)	React (este deber)
Minutos hasta primera compilación	no registrado	9.83
Minutos hasta que pasan los 6 casos	no registrado	10.15
Iteraciones del usuario después del prompt inicial	no registrado	0
Líneas escritas a mano por estudiante	no registrado	0
Enunciados spec modificados	-	0/64
Enunciados Constitution modificados	-	13/27
Plan: enunciados modificados	-	40/54
Plan: líneas físicas del diff	-	+50 / -44
Casos de aceptación que pasan	6/6 verificados en esta sesión	6/6
Suite completa	18/18 en navegador en esta sesión	36/36

Inicio: 2026-10-06 10:26:00; build: 2026-10-06 10:35:50; fin: 2026-10-06 10:36:09, UTC-5. Se tomó la ejecución inicial de planificación como equivalente al envío del prompt porque el usuario pidió todo en un solo mensaje. Los tiempos individuales de los runners no son el cronómetro. No hubo mensajes correctivos posteriores del usuario. Ver bitacora.md (bitacora.md) para el registro íntegro.

Limitación de orden: la suite Flutter completa pudo ejecutarse tras resolver Windows y CanvasKit durante el avance React; antes de planificar sí se habían verificado seis casos y LSP en Dart puro. Se informa esa desviación del orden indicado, sin atribuir un All tests passed previo que no ocurrió. Al entregar, ambas suites están comprobadas. El bloqueo nativo de Windows sigue aplicando; se usa la alternativa web reproducible.

Evidencias y checklist de entrega

- Spec inicial/final sin cambios; hashes e historial preservados.
- Análisis de spec y Constitution, plan nuevo y tasks trazables.
- analyze ejecutado; converge disponible y ejecutado, con limpieza de restos Vite en T019 y pasada final registrada.
- No nueva spec ni nuevas reglas de negocio; Node 24.19.0 cumple requisito. Vite (<https://vite.dev/guide/>) documenta Node 20.19+/22.12+; Node releases (<https://nodejs.org/en/about/previous-releases>) permite consultar soporte vigente.
- React: 36/36; build correcto; dominio sin React/DOM/data; concrete strategies solo data/main; errores impiden cálculo.
- Flutter SDD: 18/18 en Chrome; script para repetir prueba y excepción de dependencia documentados.
- Navegador real: desktop/móvil sin desbordamiento, cálculo offline y sin errores JS.
- main local y exportación para publicación GitHub. Estado real de publicación en ../publicacion.md; no se declara subida antes de verificar remoto.
- Pendiente personal: revisar el análisis y las pruebas solicitadas a mano y poder explicar las funciones.

Explicación paso a paso de la práctica

1. Qué se está demostrando

El cliente cambia de Flutter a React, pero no cambia la necesidad: dividir una cuenta con propina. SDD deja el QUÉ en spec.md y el CÓMO en plan.md, tasks.md y código. La práctica prueba esa separación con archivos idénticos, inventarios y pruebas; no la supone.

2. Cómo se tomó el laboratorio anterior

Se leyó el feature real 07_Participacion/divisor_cuenta/specs/001-dividir-cuenta de la rama sdd. Se comprobaron primero seis escenarios y LSP en Dart puro. Para completar las pruebas de pantalla se añadió el runner de desarrollo test y un script que sirve CanvasKit localmente: Windows bloquea el motor nativo y Flutter 3.47.1 tiene un fallo de rutas en el runner web. Finalmente las 18 pruebas pasaron en Chrome. La validación completa se terminó mientras avanzaba React, desviación de orden documentada en bitacora.md.

Para conservar la carpeta previa de main, la versión SDD de entrega se copió a Deberes/Deber 2/divisor_cuenta desde el commit original. La constitución original y los casos se preservan en las evidencias React.

3. Cómo se separó QUÉ de CÓMO

Se recorrieron 64 obligaciones de la spec por aparición. Se separaron ideas independientes y se excluyeron fechas, rama y explicación histórica. Cada fila de analisis_spec.md responde si sigue siendo válida en React. Todas siguen vigentes: 100% intacto. La fórmula, errores y criterios no mencionan Flutter. La constitución se analizó aparte: 14 reglas idénticas, 12 adaptadas y una reemplazada.

Esta parte y el archivo de casos fueron elaborados por el agente según tu petición; la guía pide revisarlos personalmente y escribirlos a mano. No debes declarar que escribiste manualmente el código generado. Lee las tablas y comprueba que puedes justificar sus decisiones.

4. Cómo se creó React

Se usó la plantilla React JavaScript de Vite y Node 24.19.0, con repositorio local propio en main. Se inicializó Spec Kit 1.0.13 con la integración Codex, como en Flutter. Se copió solo la spec del feature activo; no se ejecutó speckit-specify. El diff inicial y final está vacío, y los SHA256 coinciden.

Node ejecuta Vite y Vitest; npm instala librerías y ejecuta scripts. No hay backend ni base de datos. La app puede calcular sin red después de cargarse; no se promete abrirla por primera vez sin servir o tener sus archivos locales.

5. Cómo se preparó el CÓMO nuevo

La constitución conserva SOLID y cambia rutas, lenguaje y herramientas de pruebas. redondear(double) pasa a aplicar(valor). La prohibición de paquetes Flutter se reemplaza por el stack requerido; no se añaden librerías externas de estado.

speckit-plan generó plan, investigación, modelo de datos, contrato de pantalla y guía rápida. speckit-tasks creó 18 tareas por historias. speckit-analyze verificó la cobertura de los 9 FR y 4 SC; no encontró contradicciones funcionales. speckit-implement siguió las tareas. Las pruebas se escribieron antes del código y fallaron por imports ausentes, evidencia conservada. speckit-converge detectó restos de la plantilla Vite y añadió T019; se retiraron y se verificó otra vez.

6. Cómo funciona un cálculo

1. PantallaDivisor muestra los campos y envía eventos al hook; no hace cuentas.
 2. useDivisor guarda textos con useState. Mantener texto permite escribir coma decimal o abc sin que el navegador lo cambie automáticamente.
 3. numero convierte el texto completo: 100,00 -> 100; abc o vacío -> NaN. Personas exige texto entero.
 4. validarEntrada comprueba monto, personas y propina en ese orden. Devuelve null o el mensaje exacto; un error detiene el flujo antes del cálculo.
 5. El hook comprueba desbordamiento numérico para cumplir la precondition de la estrategia.
 6. calcularDivision hace $\text{monto} \times (1 + \text{propina} / 100) / \text{personas}$ y llama estrategia.aplicar.
 7. redondeoExacto aproxima al centavo; redondeoHaciaArriba usa el entero siguiente.
 8. resultado envuelve el importe. formateadorMoneda produce dos decimales con punto y sin símbolos.
 9. React vuelve a dibujar la pantalla. Al editar o cambiar de modo, el resultado anterior desaparece.
- Ejemplo: $100 \times (1 + 10/100) / 4 = 110/4 = 27.50$. Para 10/3, exacto da 3.33 y hacia arriba 4.00. Exacto no redistribuye el centavo sobrante: $3 \times 3.33 = 9.99$, tal como autoriza la spec.

7. Dónde se ve SOLID

Principio	Evidencia
SRP	cálculo, validación, formato y vista viven en módulos distintos
OCP	una estrategia nueva puede cumplir aplicar(valor) sin modificar calcularDivision
LSP	la misma función de cálculo recibe ambas estrategias; prueba aparte
ISP	el contrato tiene una sola operación aplicar
DIP	presentación recibe servicios; no importa data; domain no importa React ni DOM

main.jsx es el punto de composición. Crear objetos de valor cuenta/resultado en sus consumidores no infringe la regla: la inyección se exige a servicios y estrategias. El hook se importa estáticamente; se inyectan servicios ordinarios, no hooks.

8. Qué comprueban las pruebas

casosDePrueba.js contiene los seis objetos; division.test.js los recorre y registra un it por caso. Si hay error, comprueba mensaje exacto y que la calculadora no se invoca; si es válido, compara importe con toBeCloseTo. La prueba LSP sustituye ambas estrategias sin if de tipo.

pantalla.test.jsx verifica los tres casos obligatorios en controles accesibles y añade transiciones, coma decimal, cero, negativos, vacíos y desbordamiento. entrada.test.js comprueba límites y formato grande. La suite tiene 36 pruebas; las obligatorias son 10 (6 + 1 LSP + 3 pantalla). Chrome real verifica escritorio/móvil, cálculo con red desactivada y ausencia de errores JavaScript.

9. Cómo ejecutar y repetir

Desde PowerShell en C:\Programacion de apps\Deberes\Deber 2:

```
.\ejecutar.ps1 dev
# Abre http://localhost:5173
# En otra terminal:
.\ejecutar.ps1 test
.\ejecutar.ps1 build
.\ejecutar.ps1 check:architecture
```

El script usa npm normal o el npm local preparado. En otra máquina instala Node >= 22.12 con npm; entra a divisor_cuenta_web y ejecuta npm ci, npm test y npm run dev. En el exportado de entrega los comandos normales npm funcionan igual.

Para Flutter SDD:

```
cd .\divisor_cuenta
.\tool\probar_web.ps1
# Debe terminar All tests passed!
```

En una máquina cuyo motor nativo no esté bloqueado puedes usar flutter test. El script web detecta el SDK, prepara CanvasKit desde sus recursos originales y ejecuta las pruebas en Chrome. No cambia las pruebas ni las reglas de negocio.

10. Cómo explicar el cronómetro

Comenzó al iniciar la planificación de la Parte 5 (10:26:00 UTC-5). Primera compilación: 10:35:50, 9.83 min; siguió corriendo. Seis casos verdes: 10:36:09, 10.15 min; allí terminó. Son marcas de reloj observadas. El cronómetro incluye trabajo y herramientas intercaladas. No son los 2.34 segundos del comando build ni los 3.21 segundos del runner de dominio.

Iteraciones del usuario posteriores al prompt inicial: cero. Correcciones autónomas del agente no cuentan. Líneas manuales del estudiante: cero. Tiempo Flutter del laboratorio: no registrado; las pruebas de hoy no permiten reconstruir cuánto tardó el laboratorio.

11. Catálogo de funciones para defender la práctica

Función	Qué hace y por qué	Recibe	Devuelve y errores
cuenta	Agrupar datos inmutables, sin otra responsabilidad	monto/personas/propin a numéricos	objeto congelado; no valida
resultado	Representa la salida del negocio	importe	objeto congelado; no formatea
validarEntrada	Comprueba reglas y prioridad de errores	objeto cuenta	null o string exacto; no lanza excepciones
calcularDivision	Aplica fórmula y delega redondeo	cuenta válida y estrategia	objeto resultado; datos válidos y contrato son precondiciones
estrategia.aplicar	Contrato pequeño para sustitución	importe finito no negativo	número finito no negativo; exige precondición numérica
redondeoExacto	Crea implementación del contrato	nada	objeto con aplicar; Math.round(valor*100)/100
redondeoHaciaArriba	Crea otra implementación	nada	objeto con aplicar; Math.ceil(valor)
formateadorMoneda	Presenta dos decimales, punto y sin miles	importe finito	string; no calcula la división
numero	Convierte texto decimal completo	string	número o NaN si texto inválido/vacío
useDivisor	Mantiene estado y coordina servicios inyectados	validar/calcular/estrategias/formatear	campos, acciones, error y resultado; sin efectos de red
cambiar	Actualiza campo y elimina salida antigua	nombre de campo y texto	void; programa nuevo estado React
ejecutar	Convierte, valida y calcula si procede	nada; usa campos actuales	void; publica error o resultado; incluye desbordamiento/modo inválido
PantallaDivisor	Describe controles y salida accesible	dependencias	árbol JSX; no calcula negocio
onSubmit	Evita recarga del formulario y pide cálculo	evento del formulario	void
onChange	Transfiere texto o modo al hook	evento del control	void; resultado anterior se limpia
main.jsx	Compone servicios y monta React	elemento root del HTML	render; requiere root existente
archivos (verificador)	Recorre src y lista archivos	directorio	arreglo de rutas; errores FS si faltan rutas
verificarArquitectura (script)	Comprueba imports, composición y SRP/OCP	fuentes src	PASS o AssertionError con incumplimiento
abrir (pruebas UI)	Compone dobles/servicios para una prueba	nada	spy de cálculo, pantalla montada
ingresar (pruebas UI)	Simula entrada y clic	tres textos	void; permite verificar la salida observable
callbacks de it/it.each	Ejecutan expectativas por escenario	caso o parámetros	PASS o fallo del runner; no alteran esperado

Función	Qué hace y por qué	Recibe	Devuelve y errores
probar_web.ps1	Prepara recursos locales y corre Flutter web	ruta Chrome opcional	código de salida de pruebas; falla si SDK/navegador ausentes
ejecutar.ps1	Resuelve npm y ejecuta la acción elegida	dev/test/build/etc.	código de salida del comando; error si falta npm

No se añadieron lógica de login, historial, monedas, persistencia ni otras pantallas. Cambiar visualmente la tecnología no justificó cambiar el QUÉ.

Análisis de la especificación y de la Constitution

Origen: rama `sdd`, commit `0e14e00ef58f9146f9c0f51ff5fadd30f599412f`, feature `07_Participacion/divisor_cuenta/specs/001-dividir-cuenta`.

Este análisis lo elaboró el agente a solicitud del estudiante. La guía pide hacerlo a mano: debe revisarse y defenderse personalmente; no se presenta como trabajo manual del estudiante.

Método de conteo

Se cuentan obligaciones, reglas y criterios atómicos por aparición en el documento. Una obligación repetida en FR y Edge Cases se conserva en ambas posiciones: el denominador mide enunciados del documento, no requisitos únicos. Se separan ideas independientes dentro de una viñeta y se conserva cada escenario de entrada/salida como unidad verificable. Se excluyen título, fecha, rama histórica, estado, prioridades, razones de prioridad y notas sobre quién decidió: son metadatos o explicación, no comportamiento ni implementación. La rama `sdd` en la spec identifica su origen y no obliga a usar esa rama en React. La ausencia de red describe funcionamiento offline; npm solo usa red al instalar herramientas.

Spec

Enunciado de la spec	Tipo (QUÉ/CÓMO/MIXTO)	¿Viaja a React?	Justificación
S001 · Input: Una pantalla sin conexión.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S002 · Clarifications: Aceptar monto cero.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S003 · Clarifications: Aceptar propina cero.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S004 · Clarifications: Rechazar valores negativos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S005 · Clarifications: Permitir coma o punto decimal.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S006 · US1: Ingresar monto, personas y propina, calcular y ver el pago individual.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S007 · US1 Independent Test: Entradas válidas producen el resultado en la misma pantalla.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S008 · AC1: 100.00, 4 personas, 10%, exacto -> 27.50.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S009 · AC2: 90.00, 3 personas, 0%, exacto -> 30.00.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S010 · AC5: 10.00, 3 personas, 0%, exacto -> 3.33.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S011 · US2 Independent Test: Los errores aparecen sin resultado.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S012 · AC3: 50.00 y 0 personas -> Debe haber al menos una persona, sin resultado.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S013 · AC4: Monto abc -> Monto inválido, sin resultado.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S014 · US3 Independent Test: Cambiar de modo cambia el resultado del mismo reparto.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.

Enunciado de la spec	Tipo (QUÉ/CÓMO/MIXTO)	¿Viaja a React?	Justificación
S015 · AC6: 10.00, 3 personas, 0%, hacia arriba -> 4.00.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S016 · Edge Cases: Cero monto es válido.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S017 · Edge Cases: Cero propina es válido.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S018 · Edge Cases: Negativos son inválidos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S019 · Edge Cases: Valores no finitos son inválidos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S020 · Edge Cases: Personas debe ser un entero mayor o igual a uno.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S021 · Edge Cases: Campos vacíos se rechazan.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S022 · Edge Cases: Desbordamiento muestra El monto calculado es demasiado grande, sin resultado.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S023 · Edge Cases: Coma y punto son separadores decimales alternativos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S024 · Edge Cases: No aceptar separadores de miles.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S025 · Edge Cases: Al editar datos se oculta el resultado anterior hasta calcular.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S026 · Edge Cases: Al cambiar modo se oculta el resultado anterior hasta calcular.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S027 · FR-001: Una sola pantalla contiene tres entradas, modo y botón Calcular.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S028 · FR-002: Calcular monto $\times (1 + \text{propina} / 100) / \text{personas}$.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S029 · FR-003: Exacto redondea a centavos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.

Enunciado de la spec	Tipo (QUÉ/CÓMO/MIXTO)	¿Viaja a React?	Justificación
S030 · FR-003: Hacia arriba redondea al entero siguiente.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S031 · FR-004: Mostrar siempre dos decimales.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S032 · FR-004: Usar punto en la salida.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S033 · FR-005: Personas inválidas muestran Debe haber al menos una persona.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S034 · FR-006: Monto inválido muestra Monto inválido.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S035 · FR-006: Propina inválida muestra Propina inválida.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S036 · FR-007: Un error impide calcular.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S037 · FR-007: Un error elimina resultados previos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S038 · FR-008: Aceptar cero.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S039 · FR-008: Rechazar negativos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S040 · FR-008: Aceptar punto o coma en monto/propina.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S041 · FR-009: No usar red.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S042 · FR-009: No usar almacenamiento.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S043 · FR-009: No usar login.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S044 · FR-009: No usar historial.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.

Enunciado de la spec	Tipo (QUÉ/CÓMO/MIXTO)	¿Viaja a React?	Justificación
S045 · FR-009: No añadir monedas.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S046 · FR-009: No añadir otras pantallas.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S047 · Cuenta: Monto no negativo y finito.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S048 · Cuenta: Personas entero positivo.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S049 · Cuenta: Propina no negativa y finita.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S050 · Resultado: Importe individual después de la estrategia elegida.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S051 · SC-001: Los seis escenarios devuelven exactamente los mensajes o importes definidos.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S052 · SC-002: Ningún error conserva un resultado visible.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S053 · SC-003: Ambas reglas se seleccionan.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S054 · SC-003: Ambas reglas funcionan sin conexión.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S055 · SC-004: Cero tiene verificación ejecutable.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S056 · SC-004: Coma decimal tiene verificación ejecutable.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S057 · SC-004: Negativos tienen verificación ejecutable.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S058 · SC-004: Entradas vacías tienen verificación ejecutable.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S059 · Assumptions: Modo inicial exacto.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.

Enunciado de la spec	Tipo (QUÉ/CÓMO/MIXTO)	¿Viaja a React?	Justificación
S060 · Assumptions: Dos personas inicialmente.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S061 · Assumptions: Propina cero inicialmente.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S062 · Assumptions: Monto inicialmente vacío.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S063 · Assumptions: Validación en orden monto, personas, propina.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.
S064 · Assumptions: El redondeo a centavos no reparte sobrantes; $3 \times 3.33 = 9.99$ se acepta.	QUÉ	Intacto	Regla observable o criterio verificable independiente de Flutter.

Total: **64**; QUÉ: **64 (100%)**; CÓMO: **0 (0%)**; MIXTO: **0 (0%)**. Viaja intacto: **100%**; adaptado: **0%**; no reusable: **0%**.

Fórmula CÓMO = $0 / 64 \times 100 = 0\%$. No supera el 30%, umbral pedagógico de este deber, no una regla universal. Enunciados a modificar: ninguno. El diff inicial y final se conserva vacío; los hashes verifican también identidad byte por byte.

Constitution, evaluada por separado

Las citas originales corresponden a `.specify/memory/constitution.md` de Flutter antes de añadir el runner web. La redacción React corresponde a `.specify/memory/constitution.md` de este proyecto. No se cuentan metadatos de versión ni la nota histórica «Esta versión inicial concreta únicamente las reglas del enunciado».

Enunciado original	Regla React	Clasificación	Justificación
C01 · SRP: Cada clase tiene una razón de cambio.	Cada función o módulo tiene una razón de cambio.	adaptada en redacción	JavaScript usa funciones y módulos.
C02 · SRP: <code>CalcularDivision</code> únicamente aplica la fórmula y delega el redondeo: no valida ni formatea.	<code>calcularDivision</code> únicamente aplica la fórmula y delega el redondeo: no valida ni formatea.	adaptada en redacción	Cambia el identificador; se conserva la responsabilidad.
C03 · SRP: <code>ValidarEntrada valida y FormateadorMoneda formatea</code> .	<code>validarEntrada valida y formateadorMoneda formatea</code> .	adaptada en redacción	Identificadores JavaScript.
C04 · OCP: Una nueva regla de redondeo se agrega implementando <code>EstrategiaRedondeo</code> en un archivo nuevo.	Una nueva regla de redondeo se agrega implementando el contrato <code>estrategiaRedondeo</code> en un archivo nuevo.	adaptada en redacción	Contrato estructural en vez de interfaz Dart.
C05 · OCP: No se modifican <code>CalcularDivision</code> ni las estrategias existentes.	No se modifican <code>calcularDivision</code> ni las estrategias existentes.	adaptada en redacción	Misma extensión sin modificar consumidores.
C06 · LSP: Todas las estrategias reciben un importe finito no negativo y devuelven un importe finito no negativo.	Todas las estrategias reciben un importe finito no negativo y devuelven un importe finito no negativo.	idéntica	Precondición independiente de tecnología.

Enunciado original	Regla React	Clasificación	Justificación
C07 · LSP: El consumidor usa la interfaz sin if de tipo ni casts concretos.	El consumidor usa la interfaz sin if de tipo ni casts concretos.	idéntica	Regla de sustitución.
C08 · LSP: Una prueba debe intercambiar exacto y hacia arriba en el mismo caso de uso.	Una prueba debe intercambiar exacto y hacia arriba en el mismo caso de uso.	idéntica	Caso de prueba reusable.
C09 · ISP: EstrategiaRedondeo declara únicamente redondear(double importe).	estrategiaRedondeo declara únicamente aplicar(valor).	adaptada en redacción	Se adapta firma Dart al contrato requerido por la guía React.
C10 · ISP: No incluye validación, formateo, persistencia ni métodos ajenos al redondeo.	No incluye validación, formateo, persistencia ni métodos ajenos al redondeo.	idéntica	Interfaz pequeña.
C11 · DIP: presentation depende de domain, nunca de data.	presentation depende de domain, nunca de data.	idéntica	Dirección de dependencia.
C12 · DIP: domain no importa package:flutter ni data.	src/domain no importa react, DOM ni data; es JavaScript puro.	adaptada en redacción	Conserva aislamiento del dominio.
C13 · DIP: main.dart es el único punto que instancia implementaciones concretas y conecta dependencias.	src/main.jsx es el único punto que instancia implementaciones concretas y conecta dependencias.	adaptada en redacción	Cambia punto de composición.
C14 · DIP: Se permite instanciar objetos de valor Cuenta y Resultado donde corresponde y widgets en la presentación; la regla de composición se aplica a servicios y estrategias inyectables.	Se permite crear objetos de valor cuenta y resultado donde corresponde y elementos JSX en presentación; la regla de composición se aplica a servicios y estrategias inyectables.	adaptada en redacción	JSX sustituye widgets; no se inyectan objetos de valor.
C15 · Arquitectura: Capas obligatorias: presentation -> domain <- data.	Capas obligatorias: src/presentation -> src/domain <- src/data.	adaptada en redacción	Rutas src en lugar de lib.
C16 · Arquitectura: Dominio Dart puro.	Dominio JavaScript puro.	adaptada en redacción	Lenguaje reemplazado; aislamiento conservado.
C17 · Seguridad: No guardar secretos ni API keys.	No guardar secretos ni API keys.	idéntica	Regla independiente del framework.
C18 · Arquitectura: Una pantalla sin red ni base de datos.	Una pantalla sin red ni base de datos.	idéntica	Mismo alcance.
C19 · Dependencias: El SDK y sus dependencias generadas son la base; no agregar paquetes externos.	Usar React con Vite, Vitest, Testing Library y jsdom; sin librerías de estado externas.	reemplazada	La prohibición de paquetes Flutter impediría usar las herramientas React exigidas. Se limita explícitamente al stack autorizado.
C20 · Pruebas: Los seis criterios de aceptación se traducen a pruebas ejecutables y separadas de sus datos.	Los seis criterios de aceptación se traducen a pruebas ejecutables y separadas de sus datos.	idéntica	Datos separados del runner.
C21 · Pruebas: Incluir la prueba LSP y las tres pruebas de widget solicitadas.	Incluir la prueba LSP y las tres pruebas de pantalla solicitadas con Testing Library.	adaptada en redacción	Widgets Flutter se sustituyen por DOM.

Enunciado original	Regla React	Clasificación	Justificación
C22 · Materia: Toda función generada debe ser explicable: propósito, entrada, salida y errores.	Toda función generada debe ser explicable: propósito, entrada, salida y errores.	idéntica	Se incluye catálogo explicativo.
C23 · Calidad: Los errores de validación deben impedir el cálculo y ocultar cualquier resultado anterior.	Los errores de validación deben impedir el cálculo y ocultar cualquier resultado anterior.	idéntica	Comportamiento verificable.
C24 · Governance: La constitución rige spec, plan, tareas y código.	La constitución rige spec, plan, tareas y código.	idéntica	Mismo gobierno.
C25 · Governance: Ante un incumplimiento se corrige primero el artefacto que define la regla.	Ante un incumplimiento se corrige primero el artefacto que define la regla.	idéntica	Mismo proceso.
C26 · Governance: Revisar el cumplimiento antes y después de implementar.	Revisar el cumplimiento antes y después de implementar.	idéntica	Analyze y converge.
C27 · Governance: Cambios de principios incompatibles aumentan MAJOR; ampliaciones MINOR; aclaraciones PATCH.	Cambios de principios incompatibles aumentan MAJOR; ampliaciones MINOR; aclaraciones PATCH.	idéntica	Mismo versionado.

idéntica: **14/27 (51.85%)**; adaptada en redacción: **12/27 (44.44%)**; reemplazada: **1/27 (3.70%)**.

Reglas modificadas: adaptadas + reemplazadas. No se introduce otro principio de producto. La cobertura crítica se concreta con las pruebas de los seis casos, LSP, pantalla y las aclaraciones SC-004. La prohibición de paquetes de la constitución original ya necesitó una excepción documentada para ejecutar Flutter web; React exige reemplazarla, no ocultarla.

Inventario de decisiones del plan

Se cuenta cada decisión independiente del plan original, incluidos archivos concretos como decisiones de estructura. Se separan frases con varias decisiones. Las referencias a otros documentos no se cuentan; su contenido se evalúa en su artefacto. Las rutas agrupadas y exclusiones son decisiones distintas. El cálculo no deduplica ideas repetidas en resumen y diseño.

ID	Flutter	React	Resultado
P01	Flutter estable	React con Vite	reemplazado
P02	Una pantalla	Una pantalla	intacto
P03	setState	useState	reemplazado
P04	Dependencias por constructor	Props y argumento del hook	adaptado
P05	Cálculo puro	Cálculo puro	intacto
P06	Estrategia de redondeo intercambiable	Estrategia de redondeo intercambiable	intacto
P07	Dart 3.13.1	JavaScript ES modules / Node 24.19.0	reemplazado
P08	Dependencias SDK Flutter, sin paquetes adicionales	React, Vite y herramientas de pruebas	reemplazado
P09	Sin storage	Sin storage	intacto
P10	flutter_test widgets	Testing Library + jsdom	reemplazado
P11	test_api para dominio	Vitest para dominio	reemplazado
P12	Android y web	Navegador web	reemplazado
P13	App móvil	Página estática	reemplazado
P14	Cálculo síncrono constante	Cálculo síncrono constante	intacto
P15	Sin operaciones de red	Sin operaciones de red	intacto
P16	Offline	Offline durante uso	intacto
P17	No modificar android/ios	React no genera carpetas nativas	reemplazado
P18	No agregar paquetes	Stack npm autorizado, sin librerías de estado	reemplazado
P19	Tres entradas, dos modos y un resultado	Tres entradas, dos modos y un resultado	intacto
P20	SRP separa cálculo/validación/formato	SRP separa cálculo/validación/formato	intacto
P21	OCP usa interfaz Dart	OCP usa contrato estructural	adaptado
P22	LSP sin casts	LSP sin casts	intacto
P23	ISP un método	ISP aplicar(valor)	adaptado
P24	DIP presentación solo dominio	DIP presentación solo dominio	intacto
P25	Composición en main	Composición en main.jsx	adaptado

ID	Flutter	React	Resultado
P26	Objetos de valor y widgets en consumidores	Objetos de valor y JSX en consumidores	adaptado
P27	Dominio no importa flutter_test ni motor	Dominio no importa React, DOM ni data	adaptado
P28	No secretos	No secretos	intacto
P29	No persistencia	No persistencia	intacto
P30	Casos críticos en pruebas	Casos críticos en pruebas	intacto
P31	lib/domain/cuenta.dart	src/domain/cuenta.js	adaptado
P32	lib/domain/resultado.dart	src/domain/resultado.js	adaptado
P33	lib/domain/estrategia_redondeo.dart	src/domain/estrategiaRedondeo.js	adaptado
P34	lib/domain/calcular_division.dart	src/domain/calcularDivision.js	adaptado
P35	lib/domain/validar_entrada.dart	src/domain/validarEntrada.js	adaptado
P36	lib/data/redondeo_exacto.dart	src/data/redondeoExacto.js	adaptado
P37	lib/data/redondeo_hacia_arriba.dart	src/data/redondeoHaciaArriba.js	adaptado
P38	lib/presentation/divisor_controller.dart	src/presentation/useDivisor.js	adaptado
P39	lib/presentation/formateador_moneda.dart	src/presentation/formateadorMoneda.js	adaptado
P40	lib/presentation/pantalla_divisor.dart	src/presentation/PantallaDivisor.jsx	adaptado
P41	lib/main.dart	src/main.jsx	adaptado
P42	test/casos_de_prueba.dart	test/casosDePrueba.js	adaptado
P43	test/division_test.dart	test/division.test.js	adaptado
P44	test/pantalla_test.dart	test/pantalla.test.jsx	adaptado
P45	test/entrada_test.dart	test/entrada.test.js	adaptado
P46	tool/verificar_domain.dart	tool/verificarArquitectura.js y Vitest	reemplazado
P47	Controller convierte texto, valida, calcula y expone estado	Hook convierte texto, valida, calcula y expone estado	adaptado
P48	Pantalla recibe controller/formateador por constructor	Pantalla recibe servicios por props; hook estático	adaptado
P49	main crea validación, cálculo, estrategias, controller y formateador	main compone validación, cálculo, estrategias y formato; hook mantiene estado	adaptado
P50	ValidarEntrada devuelve String? y null válido	validarEntrada devuelve string/null	adaptado
P51	(importe * 100).round() / 100	Math.round(importe * 100) / 100	adaptado
P52	CalcularDivision devuelve Resultado sin validar/formatear	calcularDivision devuelve resultado sin validar/formatear	adaptado

ID	Flutter	React	Resultado
P53	Controller rechaza desbordamiento antes de calcular	Hook rechaza desbordamiento antes de calcular	adaptado
P54	Bloqueo flutter_tester: intentar Chrome y Dart puro	React Vitest/jsdom + Chrome real; problema Flutter documentado aparte	reemplazado

Total: 54 enunciados; modificados/adaptados: 40; intactos: 14. No se agregan como originales las nuevas decisiones React de formato grande, parseo estricto o CSS; se describen en el plan nuevo y se justifican por FR-004, entradas inválidas y presentación.

Diff físico: 50 líneas añadidas y 44 eliminadas (94 operaciones de línea); NO equivale a 94 decisiones ni a un número único de líneas reemplazadas. La guía alterna ambas métricas; se reportan separadas.